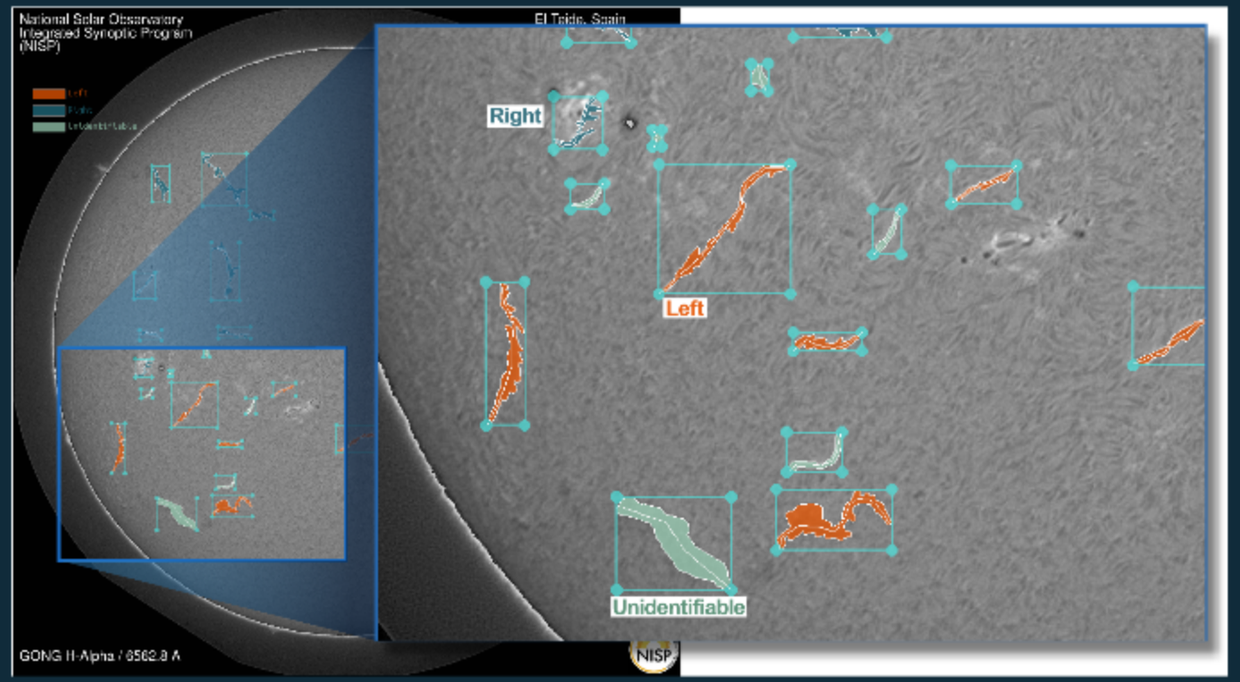
太陽画像で学ぶ ディープラーニング

Kaggleで学ぶ画像認識と領域分割

YOLO / YOLO＋SAM / U-Net



Hα観測画像と注釈：MAGFiLO

この教材で作るもの

入力：太陽を撮影したHα画像。画素の明るさを数値として扱う。

出力：フィラメント1本ごとのマスク。各画素がその1本に属するかを表す。

提出物：全テスト画像の予測をまとめた、2列のsubmission.csv。

3方式を同じ検証データで比較し、設定を決めてからテスト画像を処理する。

学習の道順

章	内容	到達点
01	画像・データ・Kaggle	入力画像と正解を確認できる
02	ニューラルネットワークと学習	重みを更新する理由が分かる
03〜05	YOLO、YOLO＋SAM、U-Net	方式ごとの入力と出力が分かる
06〜07	評価、調整、高速化	根拠を持って設定を比較できる
08	RLE、CSV、Kaggleへの提出	提出前に形式を検査できる
付録	実習手順、用語、出典	Kaggleで自分の実験を進められる

01

画像・データ・Kaggle

太陽の写真を、モデルが読める数値と正解に変える

AI・機械学習・深層学習

言葉	意味	今回の例
AI	知的な処理を計算機で実現する広い分野	画像から目的の構造を見つける
機械学習	データから判断の規則を調整する方法	画像と正解マスクの対応を学ぶ
深層学習	多くの層を持つニューラルネットワークを使う方法	YOLO、SAM、U-Net
教師あり学習	入力と正解の組を使って学習する方法	Hα画像と人が付けたマスク

画像は画素の数値表

位置	左	中央	右
上	0	64	128
中央	32	160	255
下	16	96	192

8ビットの例：0は黒、255は白。image[y, x]で位置を指定する。

グレースケール・RGB・テンソル

表現	形の例	意味
グレースケール	H × W	各位置に明るさ1個
RGB	H × W × 3	各位置に赤・緑・青の値
PyTorchの1枚	C × H × W	チャンネルを先頭に置く
ミニバッチ	B × C × H × W	B枚をまとめて計算する

画像認識の4つの課題

課題	出力	太陽画像での問い
画像分類	画像全体のラベル	フィラメントを含むか
物体検出	物体ごとの長方形	どこに何本ありそうか
セマンティック分割	画素ごとの種類	どの画素がフィラメントか
インスタンス分割	物体ごとの画素集合	どの画素が何本目に属するか

マスクに付く番号の意味

画素の位置	二値マスク	個体番号マップ
背景	0	0
フィラメントA	1	1
フィラメントB	1	2
フィラメントC	1	3

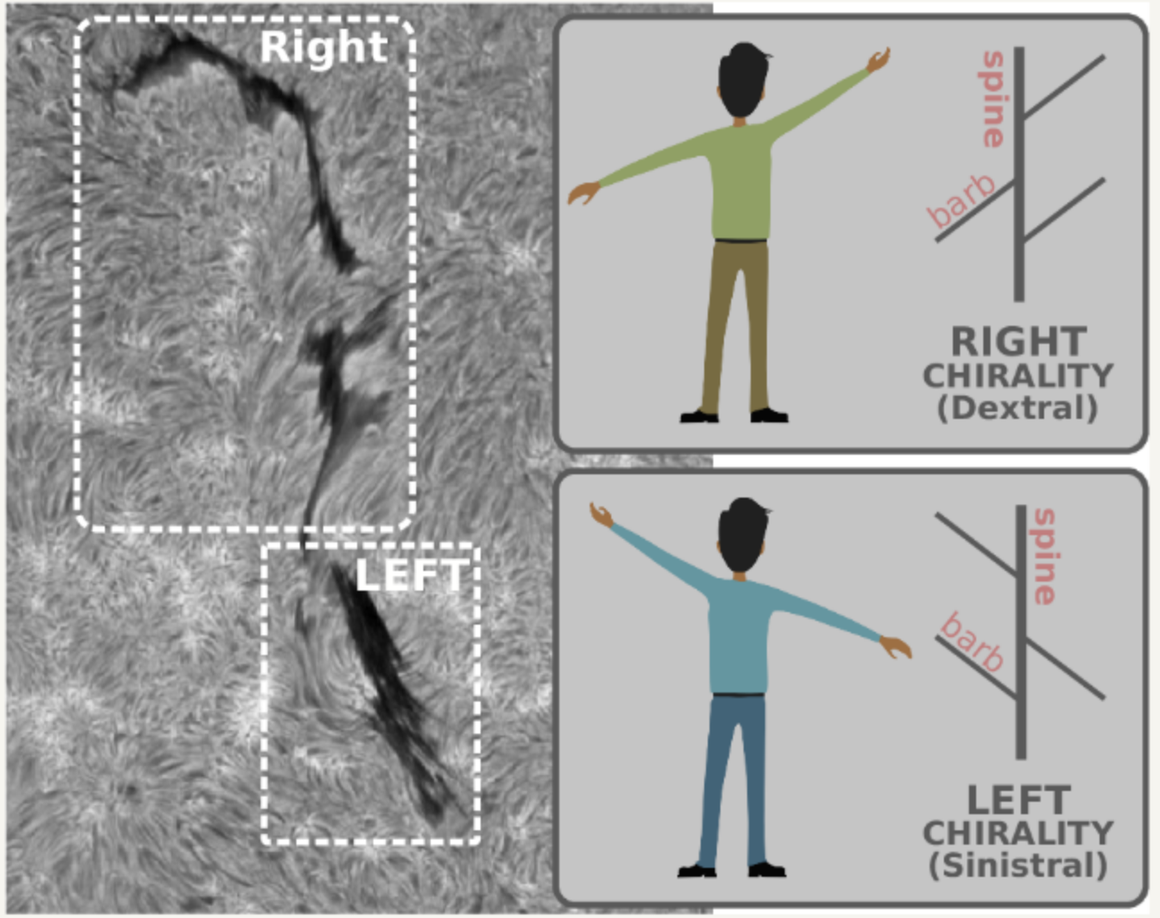
提出では、個体ごとの独立した0/1マスクを1行ずつ保存する。

Hα画像に写る太陽フィラメント

太陽面では、周囲より暗い細長い構造として見える。

主軸から伸びる細い枝をbarb（バーブ）と呼ぶ。

Left / Rightは磁気的な向きに関する分類。
画像の左右の位置とは異なる。



観測例と模式図：MAGFiLO論文 図7

太陽画像で難しいところ

細い枝：画像を小さくすると、数画素の構造が消えやすい。

低いコントラスト：背景との明るさの差が小さい。

観測ノイズ：暗い模様や欠陥をフィラメントと間違えやすい。

形のつながり：1本を分断したり、近い2本を結合したりしやすい。

大会で確認する評価とルール

項目	内容
定量評価 70%	中心はPanoptic Quality（PQ）。Dice、IoU、分断・結合の分布も確認
定性評価 30%	処理全体の説明、予測形状、コードの構成と説明
正解データ	学習に使える正解の範囲は大会ルールに従う
テスト推論	提供されたテストHα画像を使用。外部の正解メタデータを使わない
提出	1本につき1行。filament_idとsegmentation_rleの2列

共通パイプラインの全体像

順番	処理	残すもの
1〜2	Notebook設定、フォルダ・JSON確認	環境情報、データ一覧
3〜4	元画像単位で分割、前処理	固定したtrain / validation一覧
5	モデルの学習	重み、設定、学習ログ
6	検証画像を推論し、PQで比較	誤りの例、採用する設定
7〜8	設定を固定してテスト推論、元解像度へ復元	1本ごとのマスク
9	RLE化、CSV検査、提出	submission.csv

Kaggleの入力と出力フォルダ

場所	役割	例
/kaggle/input	追加したデータ。基本は読み取り用	大会の画像、JSON、追加した重み
/kaggle/working	Notebookが作成するファイル	加工ラベル、ログ、重み、CSV
train/train_images	正解と対応する学習画像	*.jpg
train/…train.json	画像情報とアノテーション	COCO形式のJSON
test/test_images	提出対象の画像	正解を推測する画像

Kaggle Notebookの作成と設定

大会ページで参加規約を確認し、CodeからNew Notebookを作る。

Inputに大会データがあることを確認する。必要ならAdd Inputで追加する。

Notebookの設定からGPUを選ぶ。利用できる種類・台数・残時間を確認する。

重みやライブラリを取得する場合はInternet設定を確認する。

コードセルを上から順に実行する。説明はMarkdownセルへ書く。

CPU・GPU・RAM・VRAM

部品	主な仕事	詰まりやすい原因
CPU	JPEG読み込み、変換、前処理	前処理が遅く、GPUがデータ待ち
GPU	多数の数値を並列に計算する	大きいモデルや画像で計算量が増加
RAM	CPU側の画像や配列を保持する	全画像をfloatでキャッシュしすぎる
VRAM	GPU側の重みや途中の計算結果を保持する	画像サイズ・バッチを増やしすぎる

ライブラリの導入とGPU確認

```
%pip install -q ultralytics pycocotools
%pip install -q segmentation-models-pytorch

import torch
import ultralytics

print(torch.__version__)
print(ultralytics.__version__)
print(torch.cuda.is_available())
print(torch.cuda.device_count())
if torch.cuda.is_available():
    print(torch.cuda.get_device_name(0))
```

%pip：
Notebookでパッケージを導入する命令。

import：
導入した機能をPythonから使う準備。

CUDA：NVIDIA
GPUを計算に利用する仕組み。

Falseや0なら、学習前にGPU設定を確認する。

実験を再現するための記録

```
from pathlib import Path
import random, json, numpy as np

SEED = 42
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)

OUT = Path('/kaggle/working/solar_run')
OUT.mkdir(parents=True, exist_ok=True)
config = dict(seed=SEED, image_size=512,
              batch_size=4, lr=3e-4)
(OUT / 'config.json').write_text(
    json.dumps(config, indent=2))
```

乱数の種（seed）は、データ分割などの再現に使う。

同じseedでも、GPU演算が完全一致するとは限らない。

設定、分割一覧、ライブラリの版、重みを一緒に残す。

大会フォルダをコードで見つける

```
INPUT = Path('/kaggle/input')
name = 'MAGFiLO_1.0_Annotations_kaggle2026_train.json'
found = sorted(INPUT.rglob(name))
assert len(found) == 1, found

ANN_PATH = found[0]
ROOT = ANN_PATH.parent.parent
TRAIN_DIR = ROOT / 'train' / 'train_images'
TEST_DIR = ROOT / 'test' / 'test_images'

assert TRAIN_DIR.is_dir()
assert TEST_DIR.is_dir()
print(ROOT)
```

rglob：
下位フォルダを含めて名前を探す。

assert：
条件が違う場合に、その場で処理を止める。

複数候補が出た場合は、中身を確認して1つを選ぶ。

JSONを読み、画像と正解を対応させる

```
from pycocotools.coco import COCO

coco = COCO(str(ANN_PATH))
image_ids = coco.getImgIds()
record = coco.loadImgs([image_ids[0]])[0]
ann_ids = coco.getAnnIds(imgIds=[record['id']])
anns = coco.loadAnns(ann_ids)

print(record)
print('instances:', len(anns))
mask = coco.annToMask(anns[0])
```

JSON：
辞書やリストを保存するテキスト形式。

images：
画像ID、ファイル名、幅、高さの情報。

annotations：
どの画像の、どの物体かを表す情報。

annToMask：
輪郭などの注釈を0/1マスクに変換する。

COCOアノテーションの読み方

キー	意味	注意点
id / image_id	注釈のID / 対象画像レコードのID	ファイル名とIDは区別する
segmentation	輪郭の点列、またはRLE	評価用の各個体を保存する
bbox	左上x、左上y、幅、高さ	右下x、右下yとは形式が違う
category_id	元データの種類ラベル	今回の実習では1クラスに統合
iscrowd / area	群集フラグ / 面積	想定外の形式を検査する

最初の1枚を必ず可視化する

```
from PIL import Image
import matplotlib.pyplot as plt

path = TRAIN_DIR / record['file_name']
image = np.asarray(Image.open(path).convert('L'))

plt.imshow(image, cmap='gray')
for ann in anns:
    m = coco.annToMask(ann)
    plt.contour(m, levels=[0.5], linewidths=0.7)
plt.axis('off')
print(image.shape, image.dtype,
      image.min(), image.max())
```

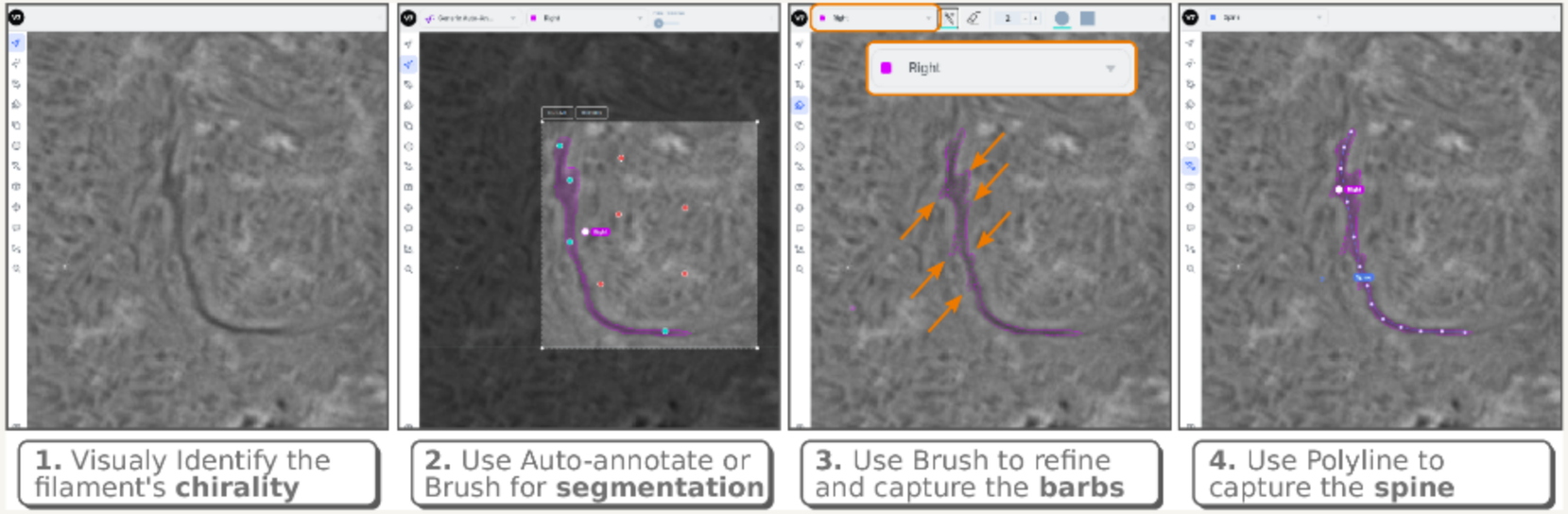
L：グレースケールとして読む。

shape：
縦横のサイズ。dtype：値の保存形式。

輪郭が暗い構造に重なるかを目で確認する。

左右反転、縦横の入れ替え、別画像との対応違いを発見する。

正解マスクを作る人の作業



専門家が画像を見て、輪郭を描き、細い枝まで修正する。

モデルは、この正解を手掛かりに学ぶ。

薄い境界では人による違いもあるため、評価方法をそろえる。

MAGFiLO論文 図4：注釈作業の例

同じ元画像に複数の注釈がある場合

元の観測画像と、注釈者ごとの画像レコードを区別する。

同じ元画像の別注釈をtrainとvalidationへ分けると、答えを見た状態になる。

検証では、注釈者ごとの独立マスクを保存し、主催者の方法で比較する。

実習のYOLO変換では元画像ごとに注釈レコードを1つ選ぶ。選択規則も記録する。

元画像単位のtrain / validation分割

```
records = coco.loadImgs(coco.getImgIds())
groups = sorted({r['file_name'] for r in records})
rng = np.random.default_rng(SEED)
rng.shuffle(groups)
n_val = max(1, round(len(groups) * 0.2))
val_names = set(groups[:n_val])

train_records = [r for r in records
                 if r['file_name'] not in val_names]
val_records = [r for r in records
               if r['file_name'] in val_names]
assert not ({r['file_name'] for r in train_records}
            & val_names)
```

80% / 20%は説明用の分割例。

同じファイル名を同じ側へまとめる。

画像が別名で重複する場合は、画素のハッシュ等でグループ化する。

近い時刻の観測も強く似るため、時間による分割も検討する。

データ漏洩を防ぐ分割の順番

処理	正しい扱い
元画像を分割	最初にtrainとvalidationを確定する
画像をタイルへ切り出す	元画像の所属を全タイルへ引き継ぐ
反転や回転で増やす	train側だけへ適用する
正規化用の統計を推定	推定が必要ならtrain側から計算する
設定を選ぶ	validationで選び、testへ適用する

学習前のデータ点検

画像数と、1枚あたりの注釈数を数える。空の画像も確認する。

画像とJSONの縦横サイズ、ファイルの存在、重複IDを検査する。

マスク面積や明るさの分布をtrainとvalidationで比べる。

背景が大半でも、フィラメントを見逃すモデルを良いと判定しない。

縮小で細い構造が消える理由

縮小率 = 512 / 2048 = 1 / 4

元画像で幅4画素の枝は、縮小後に約1画素

縦横を1/4にすると、画素数は1/16になる。

計算は軽くなるが、細い枝や境界の情報を失いやすい。

提出時に拡大しても、失った細部を完全には取り戻せない。

全体画像とタイルの使い分け

方法	長所	注意点
全体を縮小	全体の位置関係を保ちやすい	細い枝が消えやすい
高解像度の全体	形の連続性と細部を両方見られる	VRAMと時間を多く使う
重なり付きタイル	細部を大きく見せやすい	端で1本が切れ、重複予測も出る
タイル結果の統合	元画像座標へ戻してまとめる	分断・二重計数・過剰結合を検証する

明るさの正規化

x = 画素値 / 255

128 / 255 ≈ 0.502、255 / 255 = 1.0

0〜255の整数を、0〜1の浮動小数点数へ変換する。

学習時と推論時で同じ処理を使う。

事前学習モデルの推奨処理と一致するか確認する。

データ拡張は画像とマスクを同時に行う

変換	画像	正解マスク
左右・上下反転	反転する	同じ向きに反転する
回転・切り出し	位置を変える	同じ幾何変換を使う
明るさ・コントラスト	弱めに変える	位置は変えない
リサイズ	連続値として補間	二値ラベルは最近傍補間

配列がGPUへ届くまでの形

段階	shapeの例	値
PILで読む	512 × 512	uint8、0〜255
小数化・正規化	512 × 512	float32、0〜1
チャンネルを追加	1 × 512 × 512	グレースケール1枚
4枚をまとめる	4 × 1 × 512 × 512	B、C、H、W
GPUへ転送	4 × 1 × 512 × 512	形はそのまま、保存先を変更

DatasetとDataLoader

役割	すること	主な設定
Dataset	指定された1件を読み、入力と正解を返す	パス、変換、__getitem__
DataLoader	複数件をまとめてモデルへ渡す	batch_size、shuffle、num_workers
train用	毎epochの順番を変える	shuffle=True
validation用	同じ条件で比較する	shuffle=False、ランダム拡張なし

バッチ・ステップ・エポック

1 epochのstep数 = ceil(画像数 / batch_size)

100枚、batch_size=8なら ceil(100 / 8) = 13 steps

batch：一度に処理する画像のまとまり。

step：通常は1回の重み更新。

epoch：学習データを一巡すること。

3 epochsなら39回の更新。端数バッチを残す設定での例。

DataLoaderの基本設定

```
from torch.utils.data import DataLoader

train_loader = DataLoader(
    train_dataset,
    batch_size=4,
    shuffle=True,
    num_workers=2,
    pin_memory=True,
)
x, y = next(iter(train_loader))
print(x.shape, y.shape)
x = x.to('cuda', non_blocking=True)
```

train_datasetは後で定義するDatasetの実体。

pin_memoryはGPUへの転送を助ける。

non_blockingは転送を重ねるための指定。

まず2 workersで測定し、CPUとRAMを見ながら調整する。

02

ニューラルネットワークと学習

予測のずれを測り、重みを少しずつ更新する

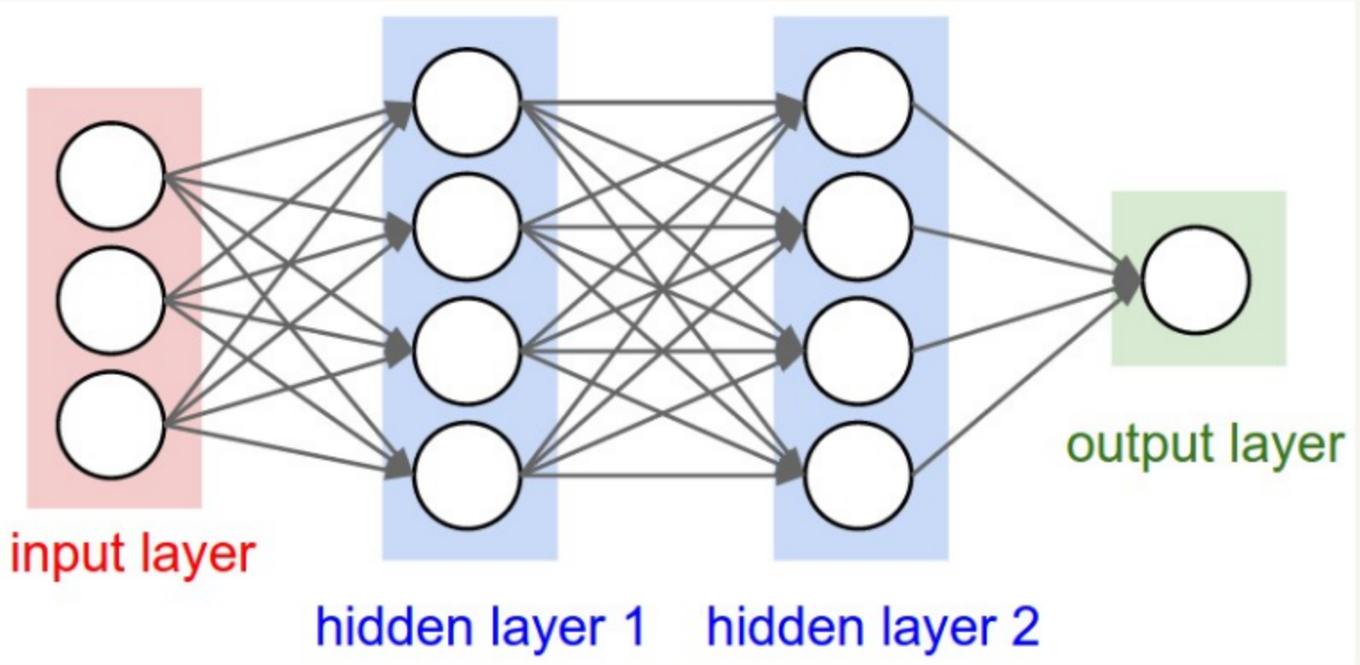
モデルは数値を変換する関数

入力xをモデルfへ渡すと、予測ŷが出る。

重みθが変わると、同じ画像に対する予測も変わる。

層を重ね、局所的な模様から広い形の特徴を作る。

学習は、正解とのずれが小さくなるようにθを調整する処理。



全結合ネットワークの例：Stanford CS231n

1つのニューロンの計算

$$z = w_1x_1 + w_2x_2 + b$$

$$0.5 \times 0.2 + (-0.3) \times 0.8 + 0.1 = -0.04$$

x_1、x_2：入力の値。

w_1、w_2：各入力をどれだけ重視するかを表す重み。

b：全体をずらすバイアス。

zを活性化関数へ入れ、次の層へ渡す。

ReLUとシグモイド

$$\mathrm{ReLU}(z) = \max(0, z)$$
$$\sigma(z) = 1 / (1 + \exp(-z))$$

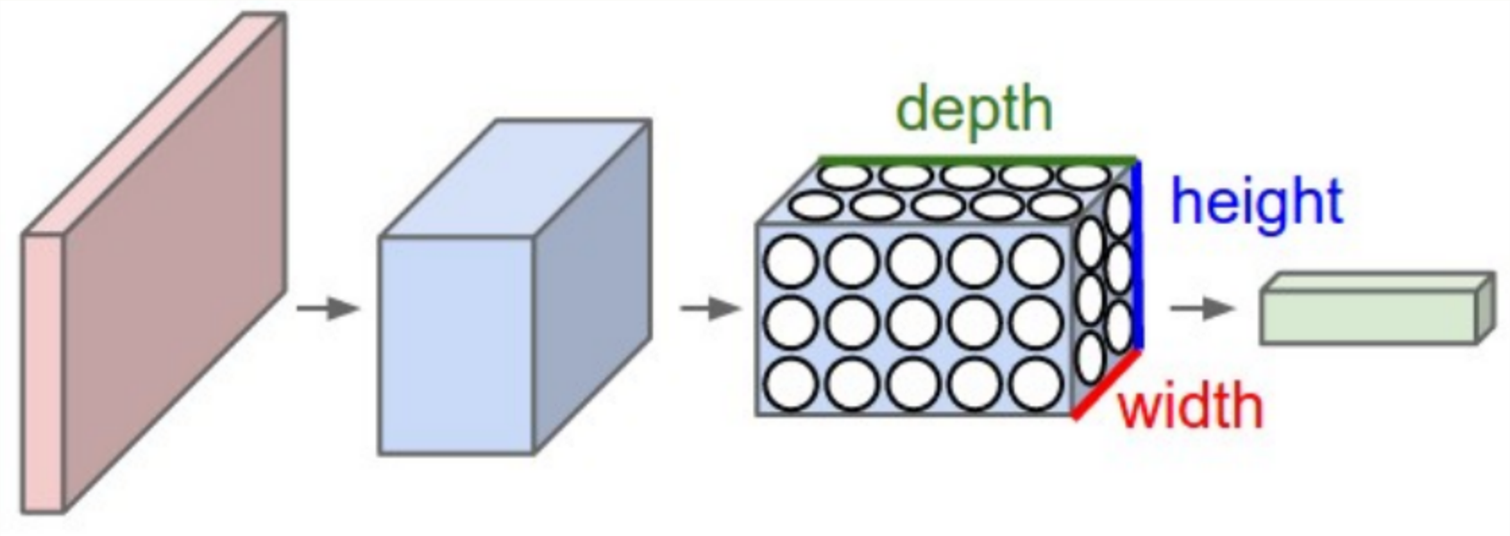
$\mathrm{ReLU}(-0.04)=0$、$\sigma(0)=0.5$、$\sigma(\ln 4)=0.8$

ReLUは負の値を0にし、途中の層で非線形な変換を作る。

シグモイドは出力を0〜1へ写し、二値マスクの確率として使う。

expは指数関数。lnは自然対数で、その逆の関係。

CNNの畳み込み



小さな重みの表（カーネル）を各位置へ当て、近くの画素の積を足す。

同じカーネルを画像全体で共有する。

複数のカーネルから複数の特徴マップを作り、線や模様の手掛かりを得る。

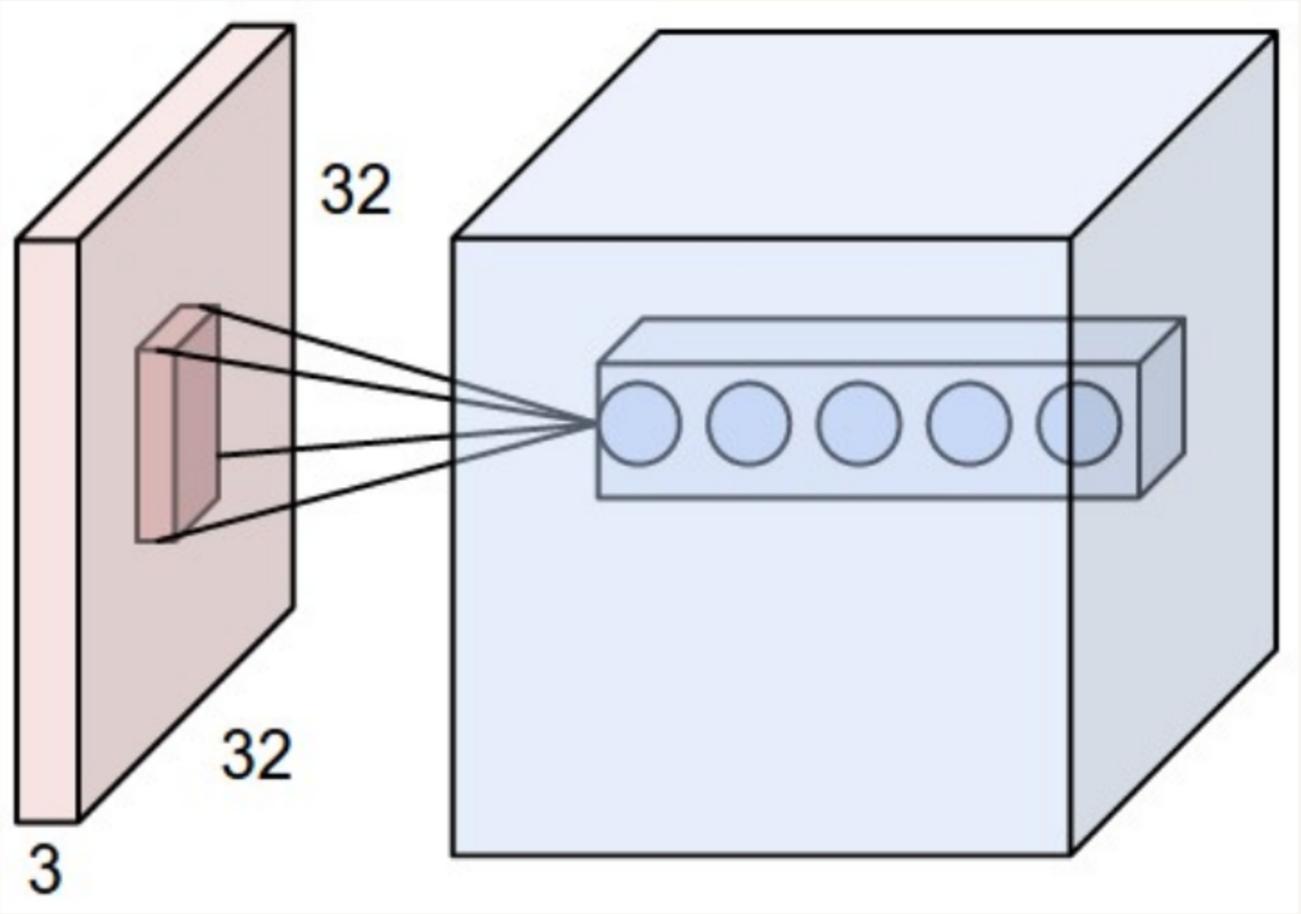
特徴マップの模式図：Stanford CS231n

カーネルが見る範囲とチャンネル

1つの出力位置は、入力の小さな領域を参照する。

RGBなら、その領域の3チャンネルを一緒に計算する。

図のdepthは特徴のチャンネル数。物体までの距離という意味ではない。



局所領域と特徴マップ：Stanford CS231n

畳み込みを手で計算する

対応位置	入力x	重みw	x × w
左上	1	1	1
右上	2	0	0
左下	3	0	0
右下	4	−1	−4
合計	バイアス0		−3

出力 = Σ(x × w) + b = 1 + 0 + 0 − 4 + 0 = −3

stride・padding・pooling

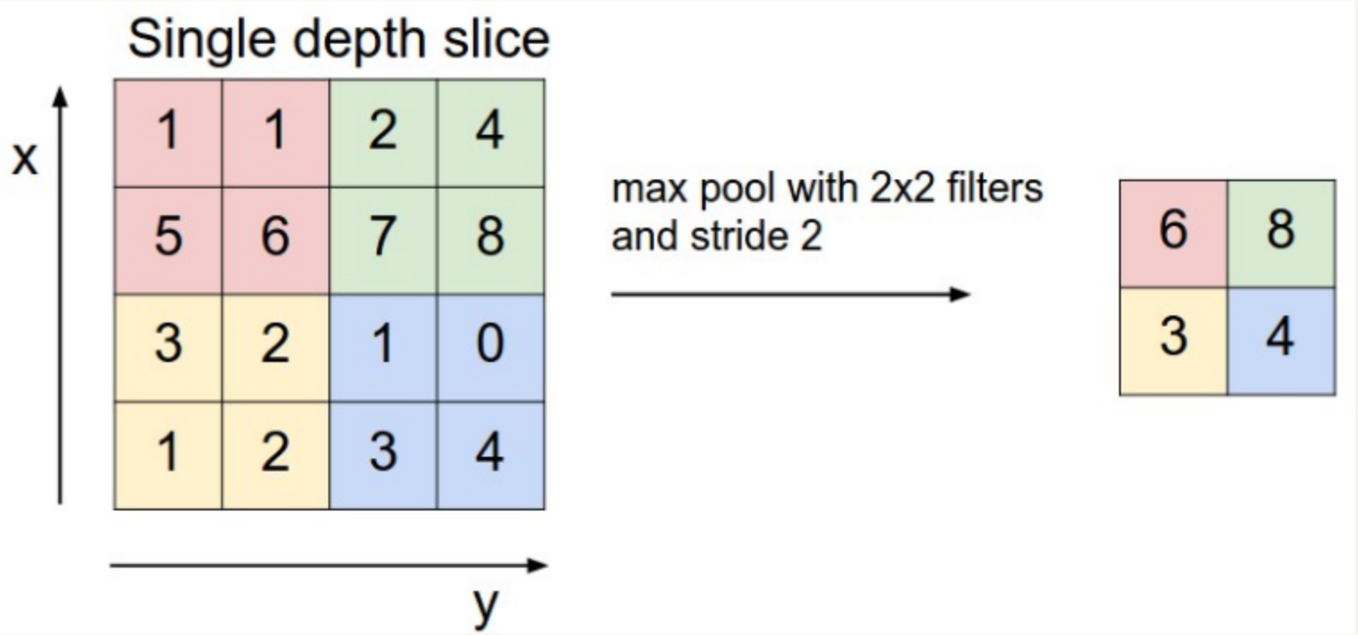
用語	意味	簡単な例
stride	カーネルを動かす間隔	2なら位置を2画素ずつ動かす
padding	周囲に値を足す処理	3×3演算でも縦横サイズを保ちやすい
max pooling	領域内の最大値を残す	[[1,2],[3,4]]の最大値は4
受容野	出力が参照する入力の範囲	層を重ねると広い形を見られる

max poolingの小さな計算例

左上の2×2領域は1、1、5、6。最大値の6を残す。

右上は8、左下は3、右下は4になる。

縦横が4から2へ小さくなる。図の軸名は原典の表記。



2×2、stride 2のmax pooling：Stanford CS231n

パラメータとハイパーパラメータ

種類	誰が決めるか	例
パラメータ	学習アルゴリズムが更新する	畳み込みの重み、バイアス
ハイパーパラメータ	実験する人が設定する	学習率、画像サイズ、batch、epoch
推論・後処理の設定	validationで選んで固定する	confidence閾値、最小面積、結合条件

1バッチの学習で起きること

順番	処理	意味
1	入力と正解をGPUへ移す	同じ計算装置上で処理する
2	forward	今の重みで予測する
3	loss	正解とのずれを1つの数へまとめる
4	backward	各重みを変えたときの損失の変化を求める
5	optimizer.step	勾配を使って重みを更新する
6	次のバッチ	新しい例で同じ処理を繰り返す

二値交差エントロピー（BCE）

$$L = -[y \ln(p) + (1-y) \ln(1-p)]$$

正解y=1：p=0.8なら $-\ln(0.8) \approx 0.223$
p=0.2なら $-\ln(0.2) \approx 1.609$

y：正解。フィラメントなら1、背景なら0。

p：モデルの予測確率。

正解へ高い確率を割り当てるほど損失が小さくなる。

Dice係数とDice損失

Dice = 2|G ∩ P| / (|G| + |P|)
Dice loss = 1 − Dice

正解10画素、予測8画素、重なり6画素
Dice = 12 / 18 ≈ 0.667、損失 ≈ 0.333

G：正解の領域。P：予測の領域。| |は画素数。

学習ではPに確率を使うsoft Diceを利用できる。

BCEと組み合わせて、画素ごとの判断と領域の重なりを学ぶ。

勾配降下法を1回計算する

$$w_{new} = w - \eta \times dL/dw$$

$L=(w-3)^2$、$w=1$なら $dL/dw=2(w-3)=-4$
$\eta=0.1$：$w_{new}=1-0.1\times(-4)=1.4$

dL/dw：重みを少し動かしたときの損失の変わり方。

η（イータ）：学習率。1回にどれだけ動かすか。

更新後の損失は2.56。更新前の4より小さくなった。

誤差逆伝播と連鎖律

$\hat{y} = w \times x$、$L = (\hat{y} - y)^2$
$dL/dw = (dL/d\hat{y}) \times (d\hat{y}/dw)$

$\hat{y}=1\times2=2$、$L=4$
$dL/dw = 2(2-4) \times 2 = -8$

入力x=2、正解y=4、重みw=1とする。

出力のずれが重みへどう伝わるかを、微分の積で求める。

学習率0.1ならw=1.8へ更新。予測3.6、損失0.16になる。

Optimizerと正則化

設定	役割	読み方
SGD	勾配に基づいて更新する	先ほどの式が基本
Adam / AdamW	勾配の履歴も使って更新幅を調整する	AdamWは重み減衰を分離して扱う
learning rate	更新量の基準	3e−4は0.0003
weight decay	大きな重みを抑える方向へ働く	過学習を抑える設定の1つ

事前学習とファインチューニング

事前学習：別の大量データで重みを先に学習しておく。

ファインチューニング：自分の画像と正解で、その重みを調整する。

太陽画像では自然画像と見た目が違うため、転用後の検証が必要。

重みを固定するfreezeと、更新するunfreezeを区別する。

過学習と汎化

過学習：trainでの成績が良くても、新しい画像で崩れる状態。

汎化：まだ見ていない観測にも対応できること。

train lossの低下だけで採用を決めず、validationのPQと形状を見る。

対策：拡張、正則化、早期終了、データ分割の見直し。

早期終了と最良の重み

epoch	validation PQ（架空）	扱い
1	0.20	最良として保存
2	0.28	最良を更新
3	0.27	改善待ち1回
4	0.26	改善待ち2回。patience=2なら停止

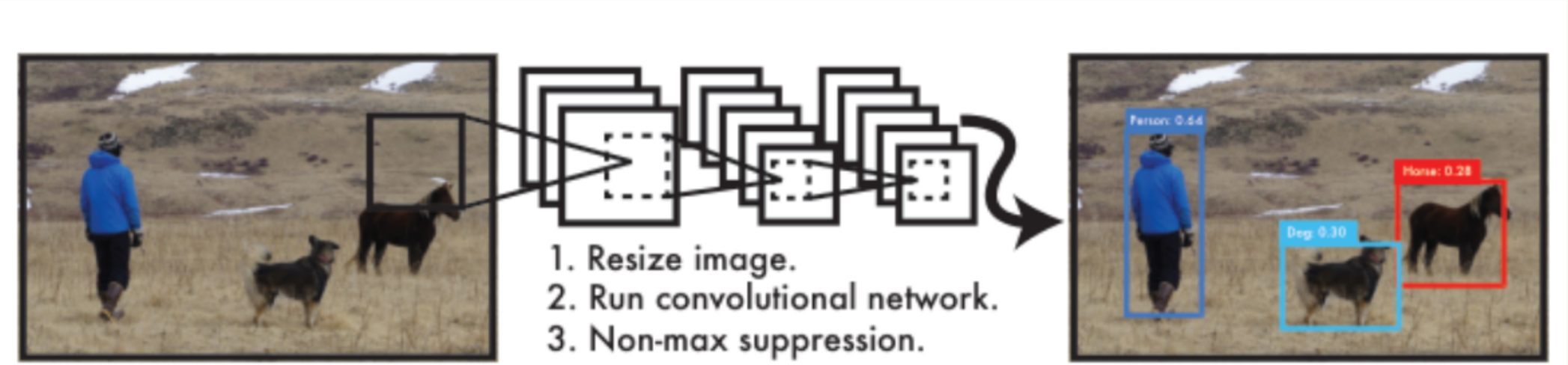
数値は説明用。監視指標と、改善判定の最小幅も記録する。

03

YOLOによる個体別マスクの予測

画像全体の特徴を使い、位置と輪郭を一度の処理で推定する

YOLOの発想



YOLOはYou Only Look Onceの略。画像から物体の位置と種類をまとめて予測する。

従来のYOLO検出は、長方形と信頼度を出す。

本教材の単独YOLO方式では、マスクも出すYOLO26-segを使う。

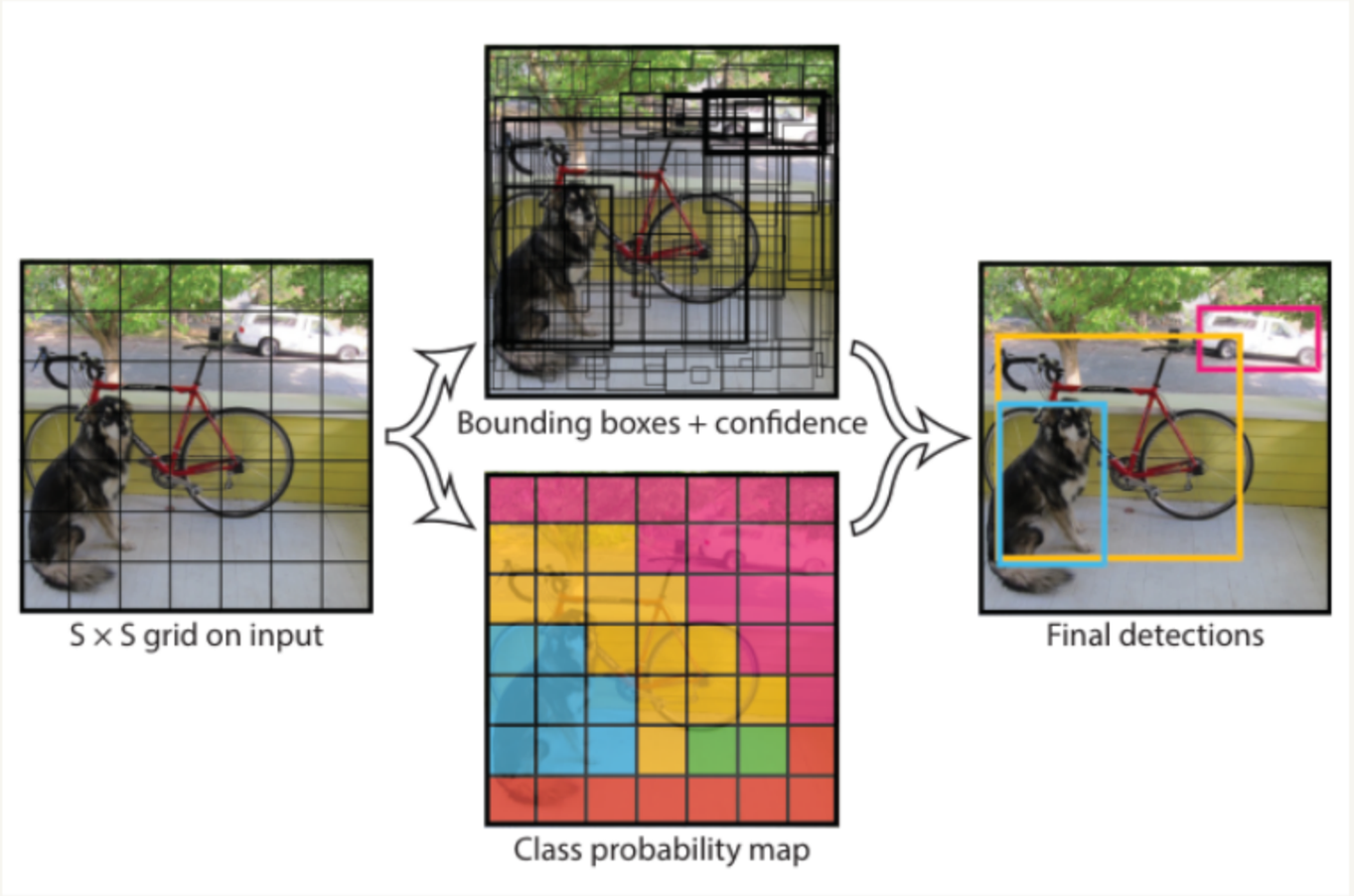
初代YOLOの説明図：Redmonら、図1。現在の実装とは構成が異なる。

グリッドと候補を使う考え方

画像の特徴から、物体がありそうな位置を複数出す。

候補ごとに位置と信頼度を推定する。

図は初代YOLOの仕組み。固定の7×7などを現行版へそのまま当てはめない。



初代YOLOのモデル図：Redmonら、図2

現行YOLOの主要な部分

部分	仕事	太陽画像での意味
Backbone	画像から特徴を抽出する	線、局所的な模様、広い形を捉える
Neck	複数の解像度の特徴を組み合わせる	小さな枝と大きな本体の情報を利用する
Detection head	位置・クラス・信頼度を出す	フィラメントがありそうな枠を出す
Segmentation head	個体ごとのマスクを出す	枠の中の実際の輪郭を推定する

検出モデルと-segモデル

項目	YOLO detection	YOLO segmentation
重みの例	yolo26n.pt	yolo26n-seg.pt
教師ラベル	クラスと長方形	クラスと輪郭の点列
推論出力	boxes	boxes と masks
今回の使用先	YOLO＋SAMの検出器	YOLO単独方式
提出マスク	別処理で作る	予測マスクを利用できる

長方形の座標を正規化する

x_center = (x_left + width/2) / W
y_center = (y_top + height/2) / H

W=H=100、bbox=(20,30,40,20)
中心=(0.4,0.4)、幅=0.4、高さ=0.2

bboxの元形式は左上x、左上y、幅、高さ。

YOLO検出ラベルは中心x、中心y、幅、高さを0〜1へ変換する。

class=0なら、行は 0 0.4 0.4 0.4 0.2。

YOLOのマスクを作る考え方

$$z_i(u,v) = \Sigma_k\, a_{ik}\, P_k(u,v)$$
$$m_i(u,v) = \sigma(z_i(u,v))$$

1画素で$P_1=2$、$P_2=-1$、係数$a_1=0.5$、$a_2=0.25$
$z=0.5\times2+0.25\times(-1)=0.75$、$\sigma(z)\approx0.679$

P_k：共有するマスクの材料（prototype）の値。

a_{ik}：物体iに合わせた組み合わせ係数。

実装では枠や拡大処理も使い、個体ごとのマスクを得る。

COCOの輪郭をYOLOラベルへ変換する

```
lines = []
for ann in annotations_for_one_image:
    parts = ann['segmentation']
    assert isinstance(parts, list) and len(parts) == 1
    xy = np.asarray(parts[0], dtype=float).reshape(-1, 2)
    assert len(xy) >= 3
    xy[:, 0] /= image_width
    xy[:, 1] /= image_height
    assert ((0 <= xy) & (xy <= 1)).all()
    text = ' '.join(f'{v:.6f}' for v in xy.ravel())
    lines.append('0 ' + text)
label_path.write_text('\n'.join(lines))
```

1行が1本。class番号は0。

xを幅で割り、yを高さで割る。

画像と同じファイル名で、拡張子を.txtにする。

1本が1輪郭の例。複数輪郭やRLEは別対応が必要。

学習フォルダとdataset.yaml

```
# /kaggle/working/yolo_seg/
# images/train/*.jpg
# images/val/*.jpg
# labels/train/*.txt
# labels/val/*.txt

path: /kaggle/working/yolo_seg
train: images/train
val: images/val
names:
  0: filament
```

上半分はフォルダ構成の説明。
下半分がYAMLファイルの中身。

YAMLは学習に必要な場所とクラス名を記述する設定形式。

画像とラベルの名前が一致しているか、変換後に確認する。

YOLO-segを学習する

```
from ultralytics import YOLO

model = YOLO('yolo26n-seg.pt')
model.train(
    data='/kaggle/working/yolo_seg/dataset.yaml',
    imgsz=512, epochs=30, batch=4,
    optimizer='AdamW', lr0=3e-4,
    patience=7, workers=2, device=0,
    hsv_h=0.0, hsv_s=0.0, hsv_v=0.05,
    mosaic=0.0, mixup=0.0,
    mask_ratio=2, overlap_mask=False,
    project=str(OUT), name='yolo_seg',
)
```

imgsz：
入力サイズ。epochs：最大の周回数。

lr0：
初期学習率。patience：改善を待つ回数。

自然画像の色変換や強い合成を弱め、まず動作を確認する。

全て初期設定の例。大会での最適値ではない。

YOLOのログの読み方

表示	意味	大会での使い方
box loss / seg loss	枠やマスクの学習損失	急増や停滞を調べる
precision / recall	検出の正確さ / 見逃しの少なさ	閾値との関係を確認する
mask mAP	複数のIoU閾値などで評価する指標	PQとは別の指標
best.pt / last.pt	内部評価上の最良 / 最新の重み	大会PQで改めて比較する
results.csv	各epochのログ	学習曲線を描いて保存する

YOLOから元画像サイズのマスクを得る

```
model = YOLO(str(OUT / 'yolo_seg/weights/best.pt'))
result = model.predict(
    source=str(image_path),
    imgsz=512, conf=0.25, iou=0.7,
    retina_masks=True, device=0,
    verbose=False,
)[0]

masks = [] if result.masks is None else (
    result.masks.data.cpu().numpy() > 0.5)
for m in masks:
    assert m.shape == (2048, 2048)
```

conf：
低い信頼度の候補を除く閾値。

iou：
NMSを使う経路での、重複候補の扱い。

retina_masks：
元画像と対応する高解像度マスクを得る指定。

検出0件はNoneを処理する。

YOLOで起きる誤りと調整候補

症状	確認すること	試す候補
細い構造の見逃し	縮小後にも見えるか	入力解像度、mask_ratio、タイル
1本が複数の候補になる	枠とマスクの重複	候補閾値、重複除去、学習ラベル
暗い模様を誤検出	trainの負例と検証画像	データ拡張、confidence
境界が粗い	教師ラベルと出力解像度	マスク解像度、モデル容量

04

YOLO＋SAMによる分割

検出器の長方形を手掛かりに、SAMが輪郭を推定する

SAMは指示を受けて領域を分ける

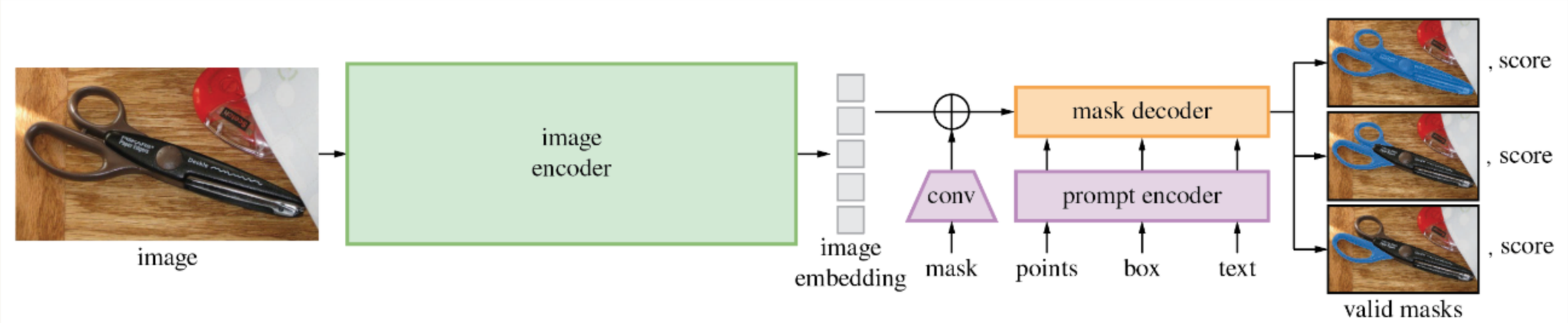


Image encoder：画像の特徴を計算する。

Prompt encoder：点や枠などの指示を数値へ変換する。

Mask decoder：特徴と指示を合わせ、候補マスクを出す。

SAMの構成図：Metaの公式リポジトリ

SAM内部の画像特徴とAttention

画像を小さな領域として扱い、各位置を特徴ベクトルへ変換する。

特徴ベクトルは、画像の模様を複数の数値で表したもの。

Attentionは、注目する位置と他の位置の関係から、集める情報の重みを計算する。

promptの位置情報を加え、decoderが「どの領域を塗るか」を推定する。

Attentionの重みづけの計算例

$$\alpha_i = \exp(s_i) / \Sigma_j \exp(s_j)$$
$$\text{出力} = \Sigma_i \alpha_i v_i$$

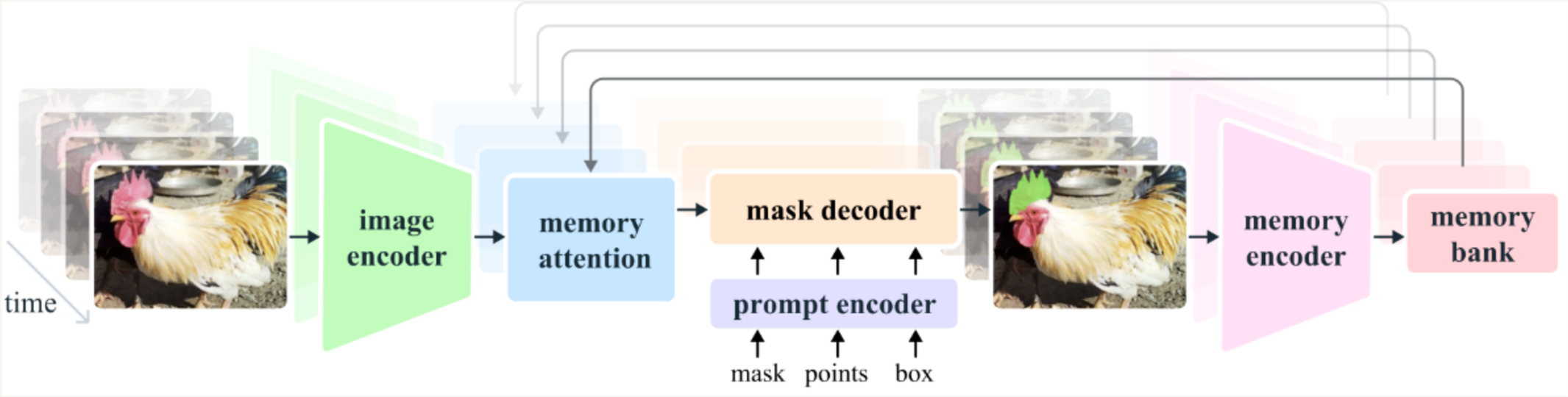
s=(ln 4, 0)なら、α=(4/5, 1/5)=(0.8, 0.2)
v=(10, 20)なら、出力=0.8×10+0.2×20=12

sは関係の強さを表すスコア。softmaxで重みの合計を1にする。

vは集める特徴。この例では分かりやすく1個の数値で表す。

実際は多次元の特徴を複数のAttentionで処理する。

SAM 2と静止画像での使い方



SAM 2は動画にも対応し、過去フレームの情報を扱うメモリを持つ。

今回の入力は1枚ずつの静止画像。動画追跡の機能は使わない。

例ではSAM 2.1 tinyの配布済み重みを固定して使う。

SAM 2の構成図：Metaの公式リポジトリ

YOLO＋SAMの処理の順番

段階	入力	出力
1 学習	学習画像と正解bbox	太陽画像用に調整したYOLO検出器
2 検出	検証またはテストの画像	枠とconfidence
3 指示	元画像上の枠	SAMへのbox prompt
4 分割	同じ画像とbox prompt	個体ごとの候補マスク
5 検証	正解マスクと候補マスク	PQ、分断・結合、処理時間

検出器用のラベルを作る

```
x, y, w, h = ann['bbox']
label = [
    (x + w / 2) / W,
    (y + h / 2) / H,
    w / W,
    h / H,
]
line = '0 ' + ' '.join(f'{v:.6f}' for v in label)

detector = YOLO('yolo26n.pt')
detector.train(data=str(DETECT_YAML),
               imgsz=512, epochs=30,
               batch=4, device=0)
```

分割用の輪郭ラベルとは別のフォルダを作る。

1行にclassと4つの枠座標を書く。

検出器の学習にも、単独YOLO方式と同じ分割を使う。

box promptの座標と余白

box = $[x_1, y_1, x_2, y_2]$
拡張後 = $[x_1-\delta, y_1-\delta, x_2+\delta, y_2+\delta]$

[100,200,150,260]、δ=10なら
[90,190,160,270]

xyxyはピクセル単位の左上と右下の座標。

画像の外へ出る値は0〜W、0〜Hへ制限する。

余白が広すぎると、隣のフィラメントを含みやすい。

YOLOの枠をSAMへ渡す

```
from ultralytics import SAM

sam = SAM('sam2.1_t.pt')
det = detector.predict(str(image_path),
        conf=0.25, imgsz=512, device=0,
        verbose=False)[0]
boxes = det.boxes.xyxy.cpu().tolist()

masks = []
if boxes:
    result = sam(str(image_path), bboxes=boxes,
                 device=0, verbose=False)[0]
    if result.masks is not None:
        masks = result.masks.data.cpu().numpy() > 0.5
```

SAMは同じ元画像と、その上の枠を受け取る。

枠が0件なら、空のリストを返す。

同じ画像にある複数の枠をまとめて渡す。

出力マスクの縦横サイズも検査する。

SAMでも残る誤り

状況	起こりうること	確認方法
枠がフィラメントを欠く	細い端や枝を取り逃す	枠の余白を小刻みに比較
枠が広すぎる	隣の暗い構造まで含める	個体ごとの輪郭を重ねて表示
複数の枠が重なる	同じ1本のマスクを二重に出す	マスクのIoUで重複候補を確認
自然画像と見た目が違う	背景や円盤の一部を選ぶ	太陽画像のvalidation PQで判断

YOLO＋SAMの調整と比較

まずYOLO検出器を固定し、confidenceと枠の余白を比較する。

同じ枠でSAMのサイズやprompt方式を比較する。

全体のPQと、検出0件・重複・過剰結合の件数を記録する。

1枚の処理時間とGPUメモリも測り、単独YOLOと比べる。

05

U-Netによる画素ごとの予測

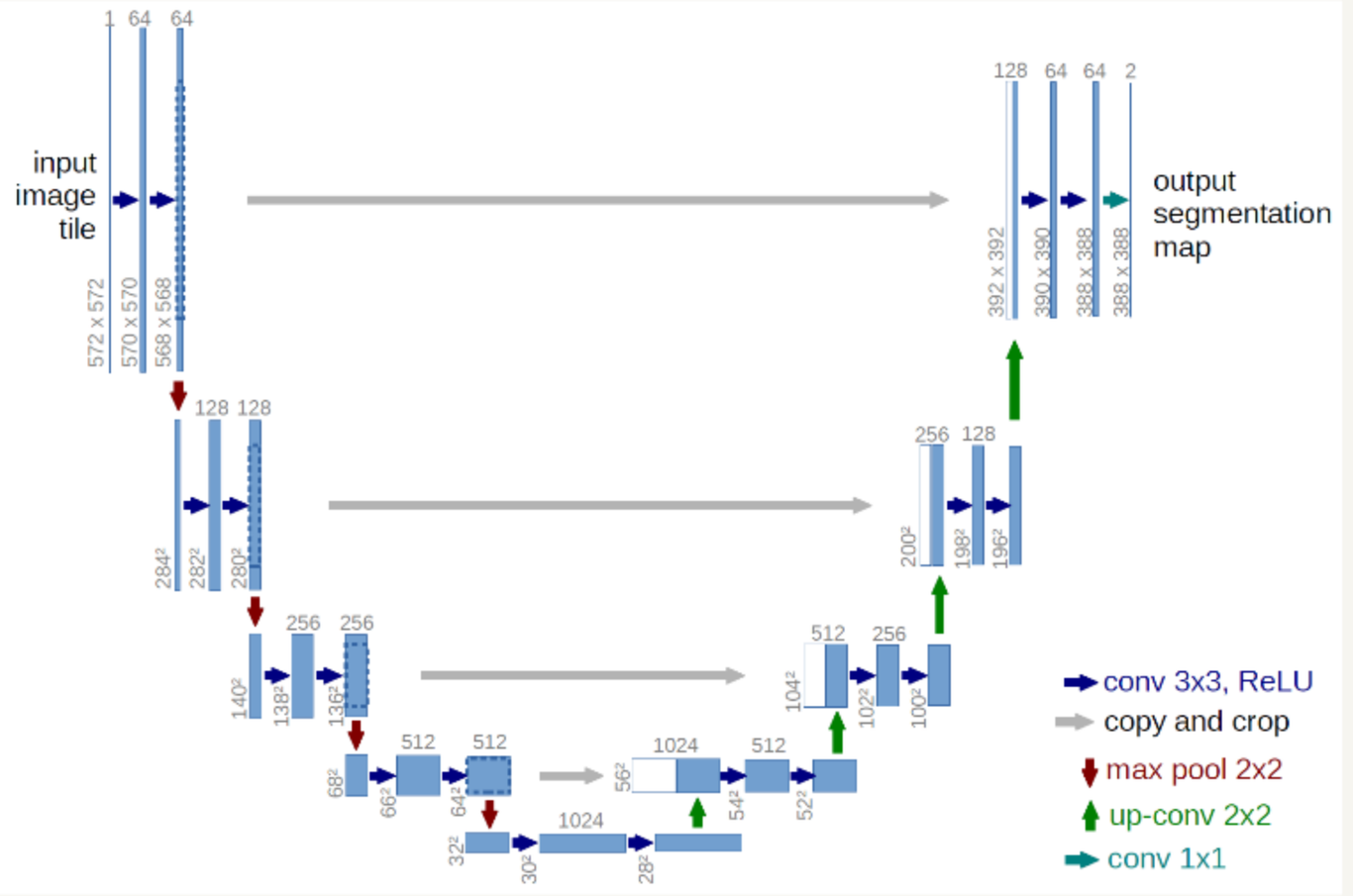
縮小で広い形を捉え、拡大しながら細部を戻す

U-NetのU字構造

左側のencoderが、解像度を下げながら特徴を抽出する。

右側のdecoderが、解像度を上げて画素ごとの出力を作る。

横のskip connectionが、細かな位置情報を右側へ渡す。



Ronnebergerらの原図。図中のサイズは2015年の原実装。

解像度とチャンネルの変化

段階	特徴の形（説明用）	役割
入力	1 × 512 × 512	元の明るさ
浅い層	32 × 512 × 512	細い線や境界
中間の層	64 × 256 × 256	局所的な形
深い層	128 × 128 × 128	広い範囲の形
出力	1 × 512 × 512	各画素のlogit

バッチ軸は省略。実際の形はmodelと入力のshapeで確認する。

skip connectionと結合

decoderの特徴とencoderの特徴を、チャンネル方向へ連結

32 × 128 × 128 と 64 × 128 × 128
連結後は 96 × 128 × 128

同じ高さ・幅の特徴を対応させる。

広い意味情報と、細い境界の位置情報を一緒に使う。

ここでの結合は足し算ではなくconcat。

ResNetの残差接続

$$y = x + F(x)$$

x=2.0、F(x)=0.3なら、y=2.3
同じ形の特徴を足すので、チャンネル数は増えない

ResNetは、途中の層へ入力を足し戻す残差接続を使う。

層は入力との差分に相当する変換F(x)を学習できる。

U-Netのチャンネル連結と区別する。足し算には形をそろえる。

U-Netの作成

```
import segmentation_models_pytorch as smp

device = torch.device('cuda' if
    torch.cuda.is_available() else 'cpu')
net = smp.Unet(
    encoder_name='resnet18',
    encoder_weights='imagenet',
    in_channels=1,
    classes=1,
    activation=None,
).to(device)

print(net(torch.zeros(2, 1, 512, 512,
                      device=device)).shape)
```

encoder：
特徴を抽出する部分。ResNet18を例に使う。

in_channels=1：
グレースケール入力。

classes=1：
前景のlogitを1個出す。

activation=None：
損失へ渡す前にsigmoidをかけない。

Datasetで入力と正解を返す

```
def __getitem__(self, index):
    r = self.records[index]
    path = TRAIN_DIR / r['file_name']
    image = Image.open(path).convert('L')
    ids = coco.getAnnIds(imgIds=[r['id']])
    anns = coco.loadAnns(ids)
    shape = (r['height'], r['width'])
    mask = np.zeros(shape, np.uint8)
    for ann in anns:
        mask |= coco.annToMask(ann)
    x = np.array(image.resize((512, 512)), np.float32)
    y = Image.fromarray(mask).resize((512, 512),
        Image.Resampling.NEAREST)
    x = torch.from_numpy(x[None].copy()) / 255
    y = torch.from_numpy(np.array(y)[None]).float()
    return x, y
```

これはDatasetクラスの中に書くメソッドの抜粋。

通常の二値U-Netの教師は、同じ注釈レコード内の前景の合体。

評価用には、合体前の個体別マスクを保持する。

BCEとsoft Diceを組み合わせる

```
import torch.nn.functional as F

def loss_fn(logits, target):
    bce = F.binary_cross_entropy_with_logits(
        logits, target)
    p = logits.sigmoid()
    axes = (1, 2, 3)
    intersection = (p * target).sum(axes)
    total = p.sum(axes) + target.sum(axes)
    dice = (2 * intersection + 1e-6) / (total + 1e-6)
    return 0.5 * bce + 0.5 * (1 - dice).mean()
```

axesはC、H、W。画像1枚ずつDiceを求める。

1e-6は0.000001。ゼロ除算を避ける小さい値。

最後にバッチ内の平均を取る。

0.5と0.5は損失の重み。これも調整対象。

U-Netの1バッチの学習

```
optimizer = torch.optim.AdamW(net.parameters(),
    lr=3e-4, weight_decay=1e-4)
scaler = torch.amp.GradScaler('cuda',
    enabled=(device.type == 'cuda'))

net.train()
for x, y in train_loader:
    x, y = x.to(device), y.to(device)
    optimizer.zero_grad(set_to_none=True)
    with torch.autocast(device_type=device.type,
            enabled=(device.type == 'cuda')):
        loss = loss_fn(net(x), y)
    scaler.scale(loss).backward()
    scaler.step(optimizer)
    scaler.update()
```

train()は学習用の動作へ切り替える。

zero_gradは前回の勾配を消す。

autocastとGradScalerで混合精度を使う。

このループをepoch数だけ繰り返す。

BatchNorm・Dropoutと検証モード

処理	学習時	検証・推論時
BatchNorm	特徴の値の分布を整えるため、バッチの統計を使う	保存した統計を使う
Dropout	一部の特徴をランダムに無効化する	無効化を止める
net.train() / net.eval()	層の動作を学習用へ切り替える	層の動作を検証用へ切り替える
inference_mode()	通常の学習では使わない	勾配の記録を止め、メモリを減らす

検証と最良重みの保存

```
net.eval()
with torch.inference_mode():
    logits = net(x_val.to(device))
    probabilities = logits.sigmoid()

# evaluate_pq is defined in the companion notebook.
metrics, _ = evaluate_pq(settings)
score = metrics['pq']
if score > best_score:
    best_score = score
    torch.save(net.state_dict(), OUT / 'best_unet.pth')
    wait = 0
else:
    wait += 1
```

eval()はBatchNormやDropoutなどの動作を検証用にする。

inference_mode()は勾配計算を省く。

保存する指標はPQなど、目的に合わせて明示する。

次のepochでは、忘れずtrain()へ戻す。

通常の二値U-Netから個体へ分ける

出力は「フィラメントらしさ」の確率画像。

確率の閾値で0/1にし、つながる画素をまとめる。

単純な連結成分法では、触れ合う2本を別々に分けられない。

切れた1本を別個体にしやすい点も、PQで確認する。

連結成分を1本ずつのマスクにする

```
import cv2

def components(probability, threshold=0.5, min_area=20):
    binary = (probability >= threshold).astype(np.uint8)
    result = cv2.connectedComponentsWithStats(
        binary, connectivity=8)
    count, labels, stats, _ = result
    instances = []
    for k in range(1, count):
        if stats[k, cv2.CC_STAT_AREA] >= min_area:
            instances.append(labels == k)
    return instances
```

8近傍は、上下左右と斜めに触れる画素をつなぐ。

番号0は背景なので除く。

min_area=20は、元画像で20画素未満を除く例。

小さい本物も消えるため、最小面積は検証で選ぶ。

個体を分ける発展法

方法	追加する情報	注意点
境界予測	個体の境目を別の出力で学ぶ	細い本体を境界として消す危険
中心とoffset	中心位置と、各画素から中心への変位	長く曲がった物体の中心設計が必要
Watershed	種から領域を広げて分ける	種が多すぎると分断が増える
近接マスクの結合	方向や距離、形状の連続性	近い別個体を結合する危険

U-Netの確率を元解像度へ戻す

```
image = Image.open(image_path).convert('L')
W, H = image.size
small = image.resize((512, 512), Image.Resampling.BILINEAR)
x = np.asarray(small, np.float32) / 255
x = torch.from_numpy(x[None, None]).to(device)

net.eval()
with torch.inference_mode():
    p = net(x).sigmoid()
    p = F.interpolate(p, size=(H, W),
        mode='bilinear', align_corners=False)
probability = p[0, 0].cpu().numpy()
masks = components(probability, 0.5, 20)
```

連続した確率を拡大し、その後に二値化する。

マスクを拡大する場合は最近傍補間を使う。

学習時と同じ入力の正規化を適用する。

06

評価と誤りの分析

画素の重なりと、1本ずつの対応を別々に確認する

画素単位のTP・FP・FN・TN

	正解がフィラメント	正解が背景
予測がフィラメント	TP：正しく見つけた画素	FP：背景を誤って塗った画素
予測が背景	FN：見逃した画素	TN：正しく背景にした画素

以降の計算例：TP=6、FP=2、FN=4。TNは重なり指標の式に入れない。

同じ領域でIoUとDiceを計算する

IoU = TP / (TP + FP + FN)
Dice = 2TP / (2TP + FP + FN)

TP=6、FP=2、FN=4
IoU=6/12=0.500、Dice=12/18≈0.667

IoUの分母は、正解と予測を合わせた領域。

Diceは重なりを2倍し、正解面積と予測面積で割る。

同じ予測でも、IoUとDiceの数値は一致しない。

Precision・Recall・F1

Precision = TP / (TP + FP)
Recall = TP / (TP + FN)
F1 = 2 × Precision × Recall / (Precision + Recall)

Precision=6/8=0.75、Recall=6/10=0.60
F1=2×0.75×0.60/1.35≈0.667

Precision：予測した中で、どれだけ正しいか。

Recall：本物のうち、どれだけ見つけたか。

同じ二値画素集合ならF1とDiceは同じ式になる。

Panoptic Quality（PQ）

PQ = Σ(対応した個体のIoU)
/ (TP + 0.5FP + 0.5FN)

対応した2本のIoUが0.8と0.7、FP=1、FN=1
PQ=(0.8+0.7)/(2+0.5+0.5)=0.500

ここでのTPは、対応が成立した個体の組。

FPは対応しない予測、FNは見逃した正解。

形の重なりと、余分な本数・見逃しの両方が影響する。

画素が合っても、個体の評価は変わる

予測の形	画素のDice	個体PQの例
10画素の1本を正確に1本で予測	1.0	IoU=1で対応。PQ=1.0
同じ10画素を5画素ずつの2本へ分断	1.0	各IoU=0.5。厳密に>0.5を満たさずPQ=0
別々の10画素の2本を20画素の1本へ結合	1.0	各IoU=0.5。どちらも対応せずPQ=0

数値は説明用。1本ごとの正解マスクを評価時に合体しない。

主催者の自己評価コードとの一致

同じ元画像の予測を、注釈者ごとの正解へそれぞれ比較する。

提供NotebookはIoU>0.5の全ペアを数える。Hungarian割当は行っていない。

対応のない予測列をFP、対応のない正解行をFNとして数える。

全レコードの分子と本数を合算してからPQを計算する。

画像ごとのPQの単純平均とは、一般に一致しない。

予測の閾値と採点基準

閾値	対象	値を上げると
confidence	YOLOの候補を残すか	低い確信の候補が減る
mask probability	U-Netの各画素を前景にするか	前景が小さくなりやすい
NMS IoU	重なる候補を重複とみなすか	NMS経路では候補を残しやすくなる
PQの対応IoU > 0.5	正解と予測の採点	大会が決める基準。自分で変更しない

validationで設定を選ぶ例

U-Netの閾値	最小面積	PQ（架空）	判断
0.30	20	0.30	薄い部分を拾うがFPも多い
0.45	20	0.34	この候補の中で採用
0.60	20	0.29	細い枝の見逃しが増えた
0.45	60	0.31	小さな本物を除いた可能性

実測結果ではない。採用値を決めた後は、testで設定を選び直さない。

失敗例を同じ形式で並べる

表示するもの	見るところ
元画像	暗い模様、ノイズ、円盤の境界
正解の個体別マスク	注釈者による違い、細い枝
予測の個体別マスク	本数、二重予測、形の連続性
重ね合わせ	位置ずれ、境界の欠落、余分な領域
数値とメモ	PQ、IoU、FP、FN、分断・結合の原因仮説

3方式の比較表の作り方

方式	直接の出力	必要な追加処理	確認点
YOLO-seg	個体別マスク	候補除去、座標復元、RLE	細部と候補の重複
YOLO＋SAM	枠を使ったマスク	検出器、prompt、重複処理	検出の見逃しと領域の選択
二値U-Net	画素の前景確率	閾値、個体化、RLE	分断と過剰結合

07

モデル調整と処理時間

原因を仮定し、少ない変更で確かめる

主なハイパーパラメータ

設定	役割	まず試す例
learning rate	重みの更新幅	1e−4、3e−4、1e−3
batch size	1回にまとめる枚数	VRAMに収まる小さな値から
epochs / patience	最大周回数 / 改善待ち	動作確認1〜2、本実験は早期終了付き
image size	入力の細かさ	512で確認、1024以上やタイルを比較
model size	表現できる量と計算量	YOLOのnからsなどへ比較

追加の調整項目

設定	変える目的	注意点
weight decay	過学習を抑える	強すぎると十分に学習できない
augmentation	観測の違いへ対応する	物理的に不自然な変換を避ける
lossの重み	画素と領域の学習の比重を変える	損失値とPQは別の尺度
mask_ratio / boundary	細部や個体境界を扱う	VRAMやラベル品質が影響する
confidence / 最小面積	予測を残す規則を変える	FPとFNの交換関係を見る

学習率スケジューラ

新しい学習率 = 今の学習率 × 係数

0.001 × 0.1 = 0.0001

学習の進行に応じて、更新幅を変える仕組み。

warmupは、初期の学習率を小さい値から徐々に増やす。

plateau型は、指標が改善しない期間の後に下げる。

早期終了とスケジューラの待ち時間の関係も確認する。

解像度とメモリの関係

画素数の比 = (1024 × 1024) / (512 × 512) = 4

同じチャンネル数なら、1枚の配列サイズは4倍

入力を大きくすると、途中の特徴マップも大きくなりやすい。

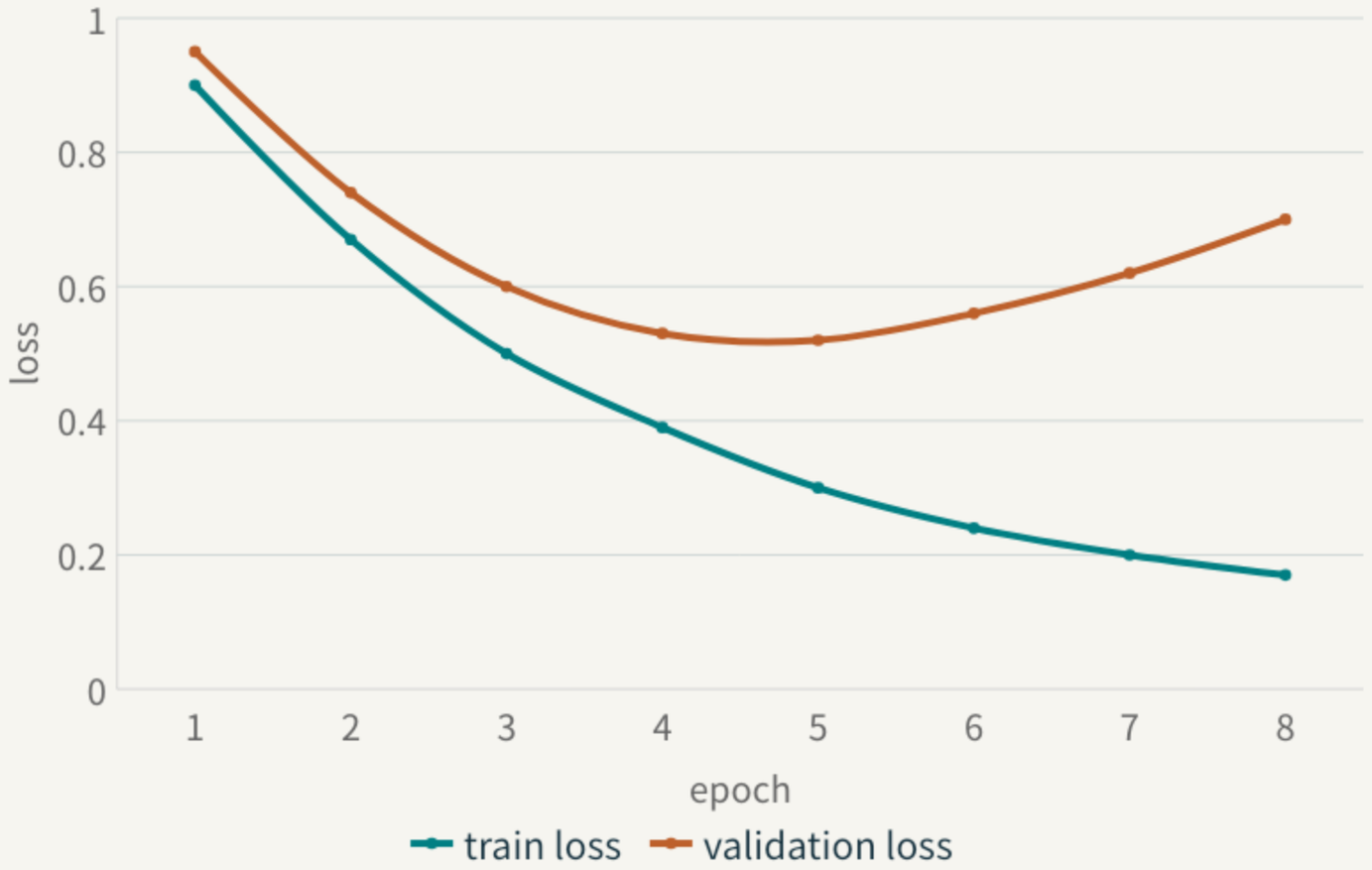
VRAM不足なら、まずbatchを下げて動作を確認する。

モデル幅、AMP、タイル、checkpointingも候補になる。

1つずつ変える実験記録

実験	変更点	validation PQ	メモ
A	基準設定	測定値を記入	時間と失敗例も保存
B	Aから画像サイズだけ変更	測定値を記入	細い枝の回復を確認
C	Aから学習率だけ変更	測定値を記入	収束の安定性を確認
D	良い重みを固定し閾値探索	測定値を記入	追加学習なしで比較

学習曲線の読み方



train lossが下がり、validation lossも下がる：学習が進む例。

trainだけ下がりvalidationが上がる：過学習や分布差を疑う。

両方ほぼ変わらない：入力・ラベル・学習率・重み更新を確認する。

説明用の架空データ。実際のモデル成績ではない。

GPUを待たせる原因の調べ方

観測	原因の候補	試すこと
GPU使用率が低い	JPEG読み込み・変換が遅い	workers、前処理キャッシュ、I/Oを測る
GPUメモリが空いている	batchが小さい	batchを少しずつ上げる
RAMが増え続ける	配列や予測を保存しすぎる	逐次処理、uint8キャッシュ、不要参照を除く
epochが長い	検証や画像保存の負荷が大きい	処理ごとに秒数を記録する

2GPUと勾配蓄積

実効batch = GPUごとのbatch × GPU数 × 蓄積回数

2 × 2 × 4 = 16枚分を、1回の更新に使う

各GPUにモデルのコピーを置き、別の画像を処理する。

VRAMは各GPUで独立しており、自動では合体しない。

勾配蓄積はlossを蓄積回数で割り、数回分の勾配を足す。

BatchNormの挙動まで、大きい単一バッチと完全一致するわけではない。

実行時間の見積もり

全体時間 ≈ epoch数 × (学習時間 + 検証時間) + 推論時間

説明例：30 × (50秒 + 20秒) + 300秒
= 2400秒 = 40分

実測した数十stepから、1epochの時間を見積もる。

データ変換、重み取得、CSV検査の時間も別に確保する。

Kaggleの残時間と保存方法を実行前に確認する。

時間切れを避ける実験の進め方

最初に2〜4枚・1epochで、RLEとCSVまで通す。

次に少数のtrainとvalidationで速度とメモリを測る。

本学習ではbestとlast、設定、分割、ログを定期的に保存する。

中断後の再開には重み以外の学習状態も必要。

検証とテスト推論を行う時間を残して、学習を終える。

08

RLEと提出CSV

元画像の各個体マスクを、規定の文字列へ変換する

3方式の出力を共通形式にそろえる

方式	途中の出力	共通の出力
YOLO-seg	候補ごとの予測マスク	list[H×Wのboolマスク]
YOLO＋SAM	枠ごとのSAMマスク	list[H×Wのboolマスク]
二値U-Net	前景確率と連結成分	list[H×Wのboolマスク]

提出前の座標とサイズ

マスクの形は2048×2048。元画像上の位置へ戻す。

リサイズ、crop、padding、タイルの位置を逆順に復元する。

確率の補間と、二値ラベルの補間を区別する。

各マスクを元画像へ重ね、位置のずれを確認する。

RLEは同じ値が続く長さを記録する

	x=0	x=1	x=2
y=0	0	1	0
y=1	0	1	0

列優先で 0,0,1,1,0,0。論理的なrun長は 2,2,2。提出文字列はpycocotoolsで作る。

COCOの圧縮RLE countsを作る

```
from pycocotools import mask as mask_utils

def encode_mask(mask):
    binary = np.asarray(mask, dtype=np.uint8)
    assert binary.shape == (2048, 2048)
    assert np.isin(binary, [0, 1]).all()
    rle = mask_utils.encode(
        np.asfortranarray(binary))
    return rle['counts'].decode('ascii')
```

uint8：
0/1を整数として保存する。

Fortran order：
列を優先するメモリ配置。

countsはbytesなので、decode で文字列にする。

提出欄にはcountsだけを入れる 。size辞書は入れない。

filament_idと行数

画像のstem	予測本数	作るID
20150125172714Mh	3	20150125172714Mh_1、_2、_3
20170501024112Bh	1	20170501024112Bh_1
別の画像	0	行を作らず、推論済み記録には残す

列名は filament_id と segmentation_rle。全テスト画像を1つのCSVへまとめる。

全テスト画像からCSVを作る

```
import pandas as pd

rows, completed = [], []
for path in test_paths:
    masks = predict_instances(path, settings)
    k = 0
    for mask in masks:
        if not np.any(mask):
            continue
        k += 1
        rows.append({
            'filament_id': f'{path.stem}_{k}',
            'segmentation_rle': encode_mask(mask)})
    completed.append(path.name)
df = pd.DataFrame(rows,
    columns=['filament_id', 'segmentation_rle'])
df.to_csv(OUT / 'submission.csv', index=False)
```

predict_instancesだけを方式ごとに切り替える。

空マスクは除く。0本の画像もcompletedへ記録する。

index=Falseで余分な番号列を付けない。

CSVは文字列値へ引用符を手動で追加せず保存する。

RLEを戻して提出内容を検査する

```
saved = pd.read_csv(OUT / 'submission.csv',
                    keep_default_na=False)
assert list(saved.columns) == [
    'filament_id', 'segmentation_rle']
assert saved['filament_id'].is_unique
assert len(completed) == len(test_paths)

counts = saved.iloc[0]['segmentation_rle']
decoded = mask_utils.decode({
    'size': [2048, 2048],
    'counts': counts.encode('ascii')})
assert decoded.shape == (2048, 2048)
assert encode_mask(decoded) == counts
```

列名とIDの重複を検査する。

全行のRLEで復元を確認する。

元マスクとの完全一致も確認する。

例は1行目の抜粋。空CSVを読む場合は先に行数を確認する。

Kaggleで保存して提出する

Notebookを新しいセッションで上から実行し、エラーなく完了させる。

Save Versionの実行・保存方法を選び、出力ファイルを残す。

Outputでsubmission.csv、重み、設定、ログを確認する。

大会の提出画面から、そのCSVを選ぶ。

形式エラーを確認し、結果と採用した設定を記録する。

よくあるエラーの切り分け

症状	確認する場所
FileNotFoundError	Input追加、パス、ファイル名、探索結果
CUDA out of memory	batch、画像サイズ、不要なGPUテンソル
lossがNaN	入力値、正解値、学習率、混合精度
masksがNone	検出0件か、-seg重みを使っているか
PQが不自然に高い	元画像の重複、GTの合体、集計方法
提出エラー	列名、画像stem、RLE形式、2048×2048

理解を確認する4問

1. YOLOの検出枠だけで、今回の提出マスクを完成できるか。

2. 同じ元画像のタイルをtrainとvalidationへ分けてよいか。

3. U-Netの前景Diceが上がっても、PQが下がるのはなぜか。

4. RLE化する前に、何画素×何画素へ戻すか。

初めての実験の到達目標

まず1枚を読み、正解の輪郭を重ねて表示する。

少数画像と1epochで、学習からCSVまで完成させる。

固定した検証集合で、3方式のPQと失敗例を比べる。

1つの原因仮説に対して1つの設定を変更する。

採用設定を固定し、全テスト画像を処理して提出する。

付録

Kaggle実習と参考資料

実行順、用語、コード例の前提、一次資料

付属Notebookの実行順

段階	すること
設定	METHODをyolo_seg、yolo_sam、unetから選ぶ
準備	Input・GPU確認、導入、パス探索、分割保存
学習	選んだ方式のセルだけ実行する
検証	同じ物理画像の全注釈でPQを計算する
選択と提出	設定を保存し、test推論とRLE検査を行う

SMOKE=Trueの出力は動作確認用。全テスト画像の提出にはSMOKE=Falseで実行する。

Notebookで使うPythonの基本

記法	意味	例
変数 = 値	値へ名前を付ける	batch_size = 4
関数(引数)	処理へ値を渡して呼び出す	encode_mask(mask)
for / if	繰り返し / 条件での分岐	各画像を処理し、空マスクは除く
def / class	関数 / データと処理の型を定義	Datasetを自作する
assert / shape	条件の検査 / 配列の形	assert mask.shape == (2048,2048)

用語集：データとモデル

用語	意味
annotation / ground truth	人が作った注釈 / 評価の基準となる正解
feature / feature map	判断の手掛かり / 位置を持つ特徴の配列
checkpoint	学習途中の重みや状態の保存
logit / probability	確率へ変換する前の値 / 0〜1の値
inference	学習した重みを使って新しい画像を予測する処理

用語集：評価と実験

用語	意味
baseline	比較の出発点となる、単純な実装や設定
validation	モデルや設定の比較に使う検証用データ
hyperparameter tuning	人が設定する値を、検証結果で調整すること
ablation	変更要素を取り除くなどして、効果を確かめる比較
seed / reproducibility	乱数の種 / 同じ条件で結果を再現できる性質

2GPUを使う発展例もKaggle内で実行

```
%%writefile /kaggle/working/train_yolo.py
from ultralytics import YOLO

if __name__ == '__main__':
    yaml = '/kaggle/working/yolo_seg/dataset.yaml'
    model = YOLO('yolo26n-seg.pt')
    model.train(data=yaml, imgsz=512, batch=8,
                workers=2, epochs=30, device=[0, 1])

# Run this in a separate notebook cell:
!python /kaggle/working/train_yolo.py
```

%%writefileはセル先頭に置く。コメントよりも前に置く。

上のスクリプトを書き出すセルと、最後の実行セルを分ける。

独自U-NetのDDPは、samplerや勾配同期も実装する。

まず1GPUで正しさを確認してから拡張する。

コード例の前提と発展課題

項目	本教材の扱い
解像度	512は動作確認用。提出マスクは元の2048へ復元
教師	1クラス分割。元画像ごとに1注釈を学習用に選択
評価	全検証注釈を独立に保持し、主催者方式のPQを使用
U-Net	通常の二値分割と連結成分。高度な個体分離は発展課題
実測成績	本教材には学習実行済みの成績を含めない

参考資料：大会とデータ

資料	用途
Kaggle 大会Overview / Data	評価、ルール、提出形式、配布ファイル
主催者 Self Evaluation Notebook	PQ集計、IoU・Dice、分断・結合の確認
Ahmadzadehら、Scientific Data、2024	MAGFiLOの構築、注釈と観測画像
Kaggle Notebooks Documentation	Notebookの作成、実行、保存

参考資料：モデルの一次資料

資料	用途
Redmonら、You Only Look Once、2016	YOLO原論文の考え方と説明図
Ultralytics YOLO26公式ドキュメント	現行モデル、学習、予測、分割ラベル
Meta SAM / SAM 2公式リポジトリ	prompt分割、構成図、SAM2の機能
Ronnebergerら、U-Net、2015	U字構造、skip connection、原図

参考資料：学習と実装

資料	用途
PyTorch 公式チュートリアル	Dataset、DataLoader、勾配とoptimizer
PyTorch AMP / DDP公式資料	混合精度、多GPU学習
Segmentation Models PyTorch公式資料	U-Net、encoderと入力・出力の指定
COCO API、pycocotools mask.py	COCO RLEのencodeとdecode
Stanford CS231n	CNN、畳み込み、poolingの解説